

Territory Conquest TCP

Relazione tecnica del progetto client-server

Progetto in C/POSIX per UNIX/Linux

Comunicazione TCP con protocollo applicativo testuale

Componenti del gruppo: Carmine Sgariglia, Massimo Russo
Repository: Client_Server_Application
Documento: relazione in formato $\text{\LaTeX}$

18 maggio 2026

Indice

1	Introduzione	2
2	Guida d'uso	2
2.1	Prerequisiti	2
2.2	Compilazione nativa	2
2.3	Avvio del server	3
2.4	Avvio del client	3
2.5	Comandi disponibili nel client	3
2.6	Esecuzione con Docker	4
3	Protocollo applicativo	4
3.1	Scelte generali	4
3.2	Messaggi client-server	5
3.3	Messaggi server-client	5
3.4	Codifica delle mappe	5
3.5	Posizioni e punteggi	6
3.6	Esempio di sessione	6
4	Architettura del software	6
4.1	Separazione dei moduli	6
4.2	Modello di concorrenza	7
5	Dettagli implementativi rilevanti	7
5.1	Framing e gestione del flusso TCP	7
5.2	Invio completo dei messaggi	8
5.3	Ciclo principale del server	8
5.4	Registrazione, login e presenza dei giocatori	9
5.5	Stabilita' degli identificatori dei giocatori	9
5.6	Regole di movimento e scoperta dei muri	10
5.7	Vista locale e vista globale	10
5.8	Client: terminale interattivo e comandi rapidi	11
5.9	Gestione della fine partita	11
6	Conformità ai requisiti	12
7	Collaudo manuale	12
8	Limiti e possibili estensioni	13
9	Conclusioni	13

1 Introduzione

Il progetto *Territory Conquest TCP* realizza un'applicazione client-server multiutente per un gioco di conquista del territorio su griglia. Il server mantiene lo stato completo della partita, gestisce registrazione e login degli utenti, assegna i giocatori alla mappa, valida i movimenti e calcola il vincitore allo scadere del tempo. I client sono programmi testuali da terminale: traducono i comandi dell'utente nel protocollo applicativo, ricevono le risposte del server e renderizzano una vista locale e una vista globale della partita.

Il protocollo di trasporto di rete è TCP, ma la parte rilevante ai fini del progetto è il protocollo al livello di applicazione definito sopra TCP. Tale protocollo è testuale, orientato a righe, e usa messaggi con prefisso **C2S** per le richieste client-server e **S2C** per le risposte server-client. La scelta di messaggi testuali rende semplice il debug, permettendo di ispezionare il traffico con strumenti standard.

La struttura del repository è divisa in tre aree principali:

- **src/common**: funzioni comuni per rete, parsing del protocollo e utilità di conversione;
- **src/server**: ciclo del server, stato del gioco, utenti e logging;
- **src/client**: ciclo del client, traduzione dei comandi utente e interfaccia testuale.

Il progetto non usa librerie esterne oltre a libc/POSIX. La compilazione nativa avviene tramite **make**; il repository contiene inoltre **Dockerfile** e **compose.yaml** per l'esecuzione containerizzata.

2 Guida d'uso

2.1 Prerequisiti

Per la compilazione nativa sono richiesti un ambiente UNIX/Linux, **gcc** e **make**. Il **Makefile** compila con lo standard C11 e abilita macro POSIX tramite:

```
-std=c11 -D_POSIX_C_SOURCE=200112L -Wall -Wextra -Wpedantic -O2 -g
```

Non sono necessarie librerie di terze parti. Per l'esecuzione via container sono richiesti Docker e, se si usa la modalità Compose, Docker Compose.

2.2 Compilazione nativa

Dalla root del progetto:

```
make
```

Il comando produce due eseguibili nella directory **bin/**:

```
bin/server
bin/client
```

Il **Makefile** separa gli oggetti comuni, server e client. I file comuni **src/common/net.c**, **src/common/protocol.c** e **src/common/utils.c** sono linkati in entrambi gli eseguibili, mentre i moduli in **src/server** e **src/client** restano specifici dei due programmi. Per eliminare artefatti di compilazione:

```
make clean
```

Sono presenti anche due target di utilità:

```
make run-server
make run-client
```

2.3 Avvio del server

Il server accetta da uno a tre argomenti:

```
./bin/server <porta> [durata_secondi] [periodo_secondi]
```

Esempio:

```
./bin/server 4242 300 5
```

Il significato degli argomenti è il seguente:

- 4242: porta TCP sulla quale il server resta in ascolto;
- 300: durata complessiva della partita in secondi;
- 5: periodo, in secondi, con cui il server invia aggiornamenti globali ai client autenticati.

Se la durata non viene specificata, il valore di default è 300 secondi. Se il periodo non viene specificato, il valore di default è 5 secondi. Il server valida la durata nell'intervallo 10–86400 secondi e il periodo nell'intervallo 1–3600 secondi. Il server non scrive su `stdout` e non legge da `stdin`.

2.4 Avvio del client

Il client richiede host e porta:

```
./bin/client <host> <porta>
```

Esempio su macchina locale:

```
./bin/client 127.0.0.1 4242
```

L'indirizzo 127.0.0.1 è corretto solo quando client e server sono eseguiti sulla stessa macchina. Se il server si trova su un host remoto, il client deve usare l'indirizzo IP o il nome DNS del server, ad esempio:

```
./bin/client 203.0.113.10 4242
```

Dopo la connessione, il client mostra una schermata testuale con stato, comandi rapidi, ultimi eventi, mappa locale e mappa globale. La mappa locale è disponibile dopo il login; la mappa globale arriva dopo il login o dopo il successivo aggiornamento periodico.

2.5 Comandi disponibili nel client

L'utente interagisce con il client tramite comandi testuali:

Comando	Effetto
register <nickname> <password>	registra un nuovo utente nel database in memoria del server
login <nickname> <password>	autentica l'utente e lo inserisce nella partita
move up down left right w, a, s, d	richiede un movimento nella direzione indicata scorciatoie per muoversi su, sinistra, giù, destra
users, list, l	richiede la lista dei giocatori online e delle loro coordinate
local	richiede la mappa locale corrente
global	richiede la mappa globale corrente
help, h	mostra nel pannello eventi un riferimento alla guida in schermata
quit, q	chiude ordinatamente la sessione

Nickname e password devono essere non vuoti, lunghi al massimo 31 caratteri e composti solo da lettere, cifre, underscore e trattino. Questa regola discende dalla funzione comune `proto_valid_name`, usata lato server in fase di registrazione.

2.6 Esecuzione con Docker

Il repository contiene un `Dockerfile` basato su Ubuntu 24.04. La build installa gli strumenti di compilazione, copia il progetto e lancia `make clean && make`. Per costruire manualmente l'immagine:

```
docker build -t territory-conquest .
```

Esempio di esecuzione manuale con rete Docker dedicata:

```
docker network create tc-net
docker run --rm --name tc-server --network tc-net -p 4242:4242 \
    territory-conquest ./bin/server 4242 300 5
docker run --rm -it --network tc-net \
    territory-conquest ./bin/client tc-server 4242
```

Con Docker Compose il server si avvia con:

```
docker compose up --build server
```

Per aprire un client:

```
docker compose run --rm client
```

3 Protocollo applicativo

3.1 Scelte generali

Il protocollo applicativo è testuale, orientato a righe e trasportato su TCP. TCP fornisce un flusso affidabile di byte, ma non conserva i confini tra messaggi. Per questo il progetto introduce un framing esplicito: ogni messaggio applicativo è una riga ASCII terminata da `\n`. La forma generale è:

```
TIPO argomento1 argomento2 ...
```

I token sono separati da spazi e non contengono spazi interni. Le strutture più complesse, come mappe e liste di giocatori, usano separatori interni diversi: `/` per separare righe, `,` per separare celle o elementi, e `:` per separare campi di uno stesso elemento.

La dimensione massima di una riga è definita da `PROTO_MAX_LINE`, pari a 16384 byte. Ogni riga viene tokenizzata con un massimo di 32 token (`PROTO_MAX_TOKENS`). Questi limiti sono dichiarati in `src/common/protocol.h` e valgono sia per client sia per server.

3.2 Messaggi client-server

I messaggi inviati dal client al server hanno prefisso C2S:

Messaggio	Semantica
C2S_REGISTER <nickname> <password>	richiesta di registrazione di un nuovo utente
C2S_LOGIN <nickname> <password>	autenticazione e ingresso nella partita
C2S_MOVE UP DOWN LEFT RIGHT	richiesta di movimento canonica
C2S_LIST_USERS	richiesta della lista dei giocatori online
C2S_LOCAL_MAP	richiesta della mappa locale del giocatore autenticato
C2S_GLOBAL_MAP	richiesta della mappa globale pubblica
C2S_QUIT	uscita ordinata dalla sessione

Il client accetta comandi più comodi per l'utente, come `register`, `login`, `move up` o `w`; poi li traduce nei messaggi canonici C2S. Ad esempio `w` diventa C2S_MOVE UP.

3.3 Messaggi server-client

I messaggi inviati dal server al client hanno prefisso S2C:

Messaggio	Semantica
S2C_OK CONNECTED	connessione accettata
S2C_OK REGISTERED	registrazione completata
S2C_OK LOGGED_IN <nick> <id> <x> <y>	login riuscito, identificatore del giocatore e posizione iniziale
S2C_OK MOVED <x> <y>	movimento eseguito e nuova posizione
S2C_OK BYE	uscita confermata
S2C_ERR <codice>	errore applicativo
S2C_LOCAL_MAP <w> <h> <mappa>	vista locale del giocatore
S2C_GLOBAL_UPDATE <w> <h> <mappa> <posizioni>	vista globale pubblica e giocatori online
S2C_USERS <posizioni>	lista dei giocatori online
S2C_GAME_OVER <winner> <score> <punteggi>	fine partita e classifica

I codici errore principali sono:

```
SERVER_FULL BAD_SYNTAX INVALID_CREDENTIALS USER_EXISTS USER_DB_FULL
AUTH_FAILED ALREADY_AUTHENTICATED USER_ALREADY_ONLINE GAME_FULL
NOT_AUTHENTICATED BAD_DIRECTION OUT_OF_BOUNDS WALL_OCCUPIED
MOVE_FAILED UNKNOWN_COMMAND LINE_TOO_LONG ENCODING_FAILED
```

3.4 Codifica delle mappe

La mappa è codificata come sequenza di celle separate da virgola, con righe separate dal carattere /. Una mappa di esempio 3x2 può essere:

```
P0,.,#/,P1,.
```

La mappa locale ha dimensione 11x11 ed è centrata sul giocatore. I simboli sono:

- @: posizione corrente del giocatore che riceve la mappa;
- #: muro scoperto da quel giocatore;

- `..`: cella libera senza proprietario noto;
- `P<n>`: cella posseduta dallo slot giocatore `n`;
- `~`: cella fuori dai confini della griglia.

I simboli precedenti appartengono al protocollo applicativo e sono quelli trasmessi dal server. Il client, nella fase di visualizzazione, li converte localmente in caratteri più leggibili per il terminale: ad esempio `@` viene mostrato come indicatore grafico del giocatore, `#` come muro pieno e le celle possedute (`P0`, `P1`, ...) come celle colorate/riempite. Questa conversione riguarda solo l'interfaccia utente e non modifica i messaggi scambiati sulla rete.

La mappa globale ha dimensione 30x15 e mostra solo le proprietà delle celle. Non rivela i muri: questa scelta è coerente con la regola di gioco per cui gli ostacoli sono informazione personale, scoperta dai singoli giocatori man mano che esplorano.

3.5 Posizioni e punteggi

Le posizioni dei giocatori online sono codificate come:

```
nickname:player_id:x:y,nickname:player_id:x:y
```

Ad esempio:

```
alice:P0:12:4,bob:P1:7:9
```

I punteggi inviati alla fine della partita sono codificati come:

```
nickname:celle,nickname:celle
```

Se non ci sono elementi da inviare, il server usa il token `-`. Tale token evita messaggi con argomenti vuoti e semplifica il parsing lato client.

3.6 Esempio di sessione

Una possibile sequenza di messaggi applicativi e':

```
S2C_OK CONNECTED
C2S_REGISTER alice secret
S2C_OK REGISTERED
C2S_LOGIN alice secret
S2C_OK LOGGED_IN alice P0 12 4
S2C_LOCAL_MAP 11 11 ...
S2C_GLOBAL_UPDATE 30 15 ... alice:P0:12:4
C2S_MOVE UP
S2C_OK MOVED 12 3
S2C_LOCAL_MAP 11 11 ...
C2S_QUIT
S2C_OK BYE
```

La sequenza mostra che il login non restituisce soltanto un esito positivo: il server comunica anche l'identificatore stabile del giocatore e la posizione iniziale. Inoltre, dopo login e movimento, il server invia una mappa locale per sincronizzare subito la vista del client.

4 Architettura del software

4.1 Separazione dei moduli

La separazione del codice segue le responsabilità principali dell'applicazione. Nel lato comune, `net.c` incapsula creazione socket, connessione e invio completo di una riga; `protocol.c` contiene validazione di nickname e password, parsing dei token e conversione delle direzioni;

`utils.c` fornisce parsing robusto di valori numerici. Questi moduli sono riutilizzati da entrambi gli eseguibili.

Nel server, `server.c` coordina socket, sessioni client, timer e dispatch dei comandi. `game.c` contiene lo stato e le regole della partita: spawn, movimento, ownership, visibilit  dei muri, mappe e vincitore. `users.c` gestisce il database utenti in memoria.

Nel client, `client.c` mantiene il ciclo eventi su socket e `stdin`, traduce input utente in protocollo e interpreta le risposte del server. `ui.c` mantiene lo stato della schermata e stampa le mappe in forma leggibile su terminale, usando colori ANSI quando `stdout`   un terminale.

4.2 Modello di concorrenza

Il server   single-process e usa `select(2)`. Non crea thread o processi figli per i client. Ogni connessione   rappresentata da una `client_session_t`, che contiene il file descriptor, lo stato di autenticazione, il `player_id`, il nickname e un buffer di input.

Questo modello ha due vantaggi nel contesto del progetto. Primo, evita problemi di sincronizzazione sullo stato globale del gioco, perch  tutte le modifiche avvengono in un unico flusso di controllo. Secondo, rende esplicito il rapporto tra eventi di rete e timer di gioco: il timeout passato a `select`   calcolato in base al prossimo aggiornamento globale o alla fine della partita.

Il client usa la stessa primitiva per monitorare contemporaneamente il socket e `stdin`. In questo modo pu  ricevere aggiornamenti globali anche mentre l'utente sta digitando un comando.

5 Dettagli implementativi rilevanti

5.1 Framing e gestione del flusso TCP

TCP   uno stream di byte: una `recv` pu  restituire una riga parziale, pi  righe insieme o una porzione arbitraria dei dati inviati. Per questo il server conserva, per ogni sessione, un buffer incrementale. Ogni volta che legge nuovi byte, li accoda a `inbuf`; poi cerca il carattere `\n` e processa tutte le righe complete trovate.

Listing 1: Estrazione di righe complete da uno stream TCP nel server.

```
1 while (1)
2 {
3     char *nl = memchr(c->inbuf, '\n', c->inbuf_len);
4     size_t line_len;
5     char line[PROTO_MAX_LINE];
6     if (nl == NULL)
7     {
8         break;
9     }
10    line_len = (size_t)(nl - c->inbuf);
11    if (line_len >= sizeof(line))
12    {
13        disconnect_client(s, index);
14        return -1;
15    }
16    memcpy(line, c->inbuf, line_len);
17    line[line_len] = '\0';
18    memmove(c->inbuf, nl + 1, c->inbuf_len - line_len - 1);
19    c->inbuf_len -= line_len + 1;
20    c->inbuf[c->inbuf_len] = '\0';
21    handle_line(s, index, line);
22    if (c->fd < 0)
23    {
24        return -1;
```



```

25     }
26 }

```

Lo stesso principio è applicato nel client per le risposte del server. Il limite `PROTO_MAX_LINE` protegge da righe eccessivamente lunghe: quando il buffer non può contenere altri dati, il server risponde con `S2C_ERR LINE_TOO_LONG` e chiude la sessione.

5.2 Invio completo dei messaggi

Anche `send` può scrivere solo una parte dei byte richiesti. La funzione comune `net_send_all` ripete l'invio finché l'intero buffer è stato trasmesso, gestendo `EINTR`. Tutti i messaggi applicativi passano poi da `net_send_line`.

Listing 2: Invio completo di una riga su socket.

```

1 static int net_send_all(int fd, const char *buf, size_t len)
2 {
3     size_t sent = 0;
4
5     while (sent < len)
6     {
7         ssize_t n = send(fd, buf + sent, len - sent, 0);
8         if (n < 0)
9         {
10             if (errno == EINTR)
11             {
12                 continue;
13             }
14             net_set_error("send su fd=%d fallita: %s", fd, strerror(errno));
15             return -1;
16         }
17         if (n == 0)
18         {
19             net_set_error("send su fd=%d ha scritto 0 byte", fd);
20             return -1;
21         }
22         sent += (size_t)n;
23     }
24     return 0;
25 }

```

Questa scelta rende più robusta la comunicazione rispetto a un invio singolo, specialmente quando il payload contiene mappe globali più lunghe.

5.3 Ciclo principale del server

Il server costruisce a ogni iterazione il set di descrittori da passare a `select`: il socket di ascolto e tutti i socket client attivi. Il timeout non è fisso, ma viene calcolato in base al prossimo evento temporizzato: aggiornamento globale o fine partita.

Listing 3: Uso di `select` nel ciclo principale del server.

```

1 FD_ZERO(&rfd);
2 FD_SET(s.listen_fd, &rfd);
3 for (i = 0; i < s.client_count; ++i)
4 {
5     if (s.clients[i].fd >= 0)
6     {
7         FD_SET(s.clients[i].fd, &rfd);

```

```

8         if (s.clients[i].fd > maxfd)
9         {
10             maxfd = s.clients[i].fd;
11         }
12     }
13 }
14
15 tv.tv_sec = seconds_until_next_event(&s);
16 tv.tv_usec = 0;
17 rc = select(maxfd + 1, &rfd, NULL, NULL, &tv);

```

Dopo il ritorno di `select`, il server accetta nuove connessioni se il socket di ascolto è pronto, legge i client pronti e poi controlla i timer. Se scade il periodo di aggiornamento, invia una `S2C_GLOBAL_UPDATE` a tutti i client autenticati. Se scade la durata totale, invia `S2C_GAME_OVER` e termina il ciclo.

5.4 Registrazione, login e presenza dei giocatori

Gli utenti sono memorizzati in un array dinamico in memoria. La registrazione verifica sintassi e unicità del nickname; il login verifica password, evita login duplicati sullo stesso socket e impedisce che lo stesso nickname sia online in due sessioni contemporaneamente.

Listing 4: Controlli principali del login lato server.

```

1 if (users_authenticate(&s->users, tok[1], tok[2]) != 0)
2 {
3     sendf(c, "S2C_ERR AUTH_FAILED");
4     return;
5 }
6 if (game_find_player(&s->game, tok[1]) >= 0)
7 {
8     sendf(c, "S2C_ERR USER_ALREADY_ONLINE");
9     return;
10 }
11 player_id = game_add_player(&s->game, tok[1]);
12 if (player_id < 0)
13 {
14     sendf(c, "S2C_ERR GAME_FULL");
15     return;
16 }
17 c->authenticated = 1;
18 c->player_id = player_id;

```

Il database utenti non è persistente: se il processo server viene terminato, registrazioni e password vengono perse. Questa è una scelta coerente con il vincolo di non usare database esterni.

5.5 Stabilità degli identificatori dei giocatori

Un dettaglio importante riguarda gli identificatori `P0`, `P1`, ... usati nelle mappe. Il codice distingue tra slot *usato* e giocatore *attivo*. Quando un client si disconnette, il giocatore diventa offline, ma lo slot non viene cancellato: le celle conquistate restano assegnate allo stesso indice. Se lo stesso nickname rientra più tardi, il server può riattivare lo slot già associato a quel nickname.

Listing 5: Riattivazione o assegnazione dello slot giocatore.

```

1 for (i = 0; i < game->player_count; ++i) {
2     if (game->players[i].used && strcmp(game->players[i].nickname, nickname) == 0) {
3         slot = i;

```

```

4         found_slot = 1;
5         break;
6     }
7 }
8 if (!game->players[slot].used) {
9     memset(&game->players[slot], 0, sizeof(game->players[slot]));
10    strncpy(game->players[slot].nickname, nickname, NICK_MAX);
11    symbol_for_slot(slot, game->players[slot].symbol, sizeof(game->players[slot].symbol));
12    game->players[slot].used = 1;
13 }
14 game->players[slot].active = 1;

```

Questa scelta evita ambiguità nella mappa globale. Una cella marcata come P0 resta di proprietà dello stesso slot logico, anche se il giocatore si scollega e poi si ricollega.

5.6 Regole di movimento e scoperta dei muri

La funzione `game_move` implementa le regole fondamentali di gioco. Il server calcola la cella di destinazione, verifica che sia dentro la griglia, controlla che non sia un muro e impedisce di entrare in una cella occupata da un altro giocatore attivo. Solo se tutti i controlli riescono aggiorna posizione e proprietà della cella.

Listing 6: Validazione del movimento e aggiornamento della proprietà.

```

1 if (!in_bounds(nx, ny)) {
2     return -2;
3 }
4 if (game->wall[ny][nx]) {
5     p->discovered_walls[ny][nx] = 1;
6     game_reveal_around(game, player_id);
7     return -3;
8 }
9 for (i = 0; i < game->player_count; ++i) {
10     if ((int)i != player_id && game->players[i].active &&
11         game->players[i].x == nx && game->players[i].y == ny) {
12         return -4;
13     }
14 }
15 p->x = nx;
16 p->y = ny;
17 game->owner[ny][nx] = player_id;
18 game_reveal_around(game, player_id);

```

Il ritorno della funzione è poi tradotto in codici applicativi: `OUT_OF_BOUNDS`, `WALL`, `OCCUPIED` o `MOVE_FAILED`. In caso di muro, il giocatore non si muove ma il muro viene segnato come scoperto nella sua matrice personale. In caso di movimento riuscito, la cella di arrivo viene assegnata al giocatore.

5.7 Vista locale e vista globale

Il server mantiene due tipi di informazione: la proprietà pubblica delle celle e la conoscenza privata dei muri. Ogni `player_t` contiene una matrice `discovered_walls[MAP_H][MAP_W]`. La mappa locale è costruita come finestra 11x11 centrata sul giocatore. Per ogni cella della finestra, il server decide se inviare ~, @, #, un simbolo proprietario o ..

Listing 7: Costruzione della mappa locale.

```

1 start_x = p->x - LOCAL_VIEW_W / 2;

```

```

2 start_y = p->y - LOCAL_VIEW_H / 2;
3
4 for (y = 0; y < LOCAL_VIEW_H; ++y) {
5     int first_cell = 1;
6     if (y > 0) {
7         append_text(out, out_size, "/");
8     }
9     for (x = 0; x < LOCAL_VIEW_W; ++x) {
10        map_x = start_x + x;
11        map_y = start_y + y;
12        if (!in_bounds(map_x, map_y)) {
13            append_cell(out, out_size, &first_cell, "~");
14        } else if (p->x == map_x && p->y == map_y) {
15            append_cell(out, out_size, &first_cell, "@");
16        } else if (p->discovered_walls[map_y][map_x]) {
17            append_cell(out, out_size, &first_cell, "#");
18        }
19    }
20 }

```

La mappa globale, invece, scorre l'intera griglia 30x15 e codifica solo `owner[y][x]`. I muri non vengono consultati nella costruzione della globale: anche se una cella è un muro, la sua natura non viene rivelata dal messaggio pubblico. Questo soddisfa la distinzione tra informazione pubblica (territori) e informazione privata (ostacoli scoperti).

5.8 Client: terminale interattivo e comandi rapidi

Il client mette il terminale in modalità non canonica quando `stdin` e `stdout` sono TTY. In questa modalità legge un byte alla volta e può trasformare subito `w`, `a`, `s`, `d` in movimenti, senza attendere il tasto Invio. Se l'input arriva da pipe o file, il client funziona comunque in modalità riga.

Listing 8: Traduzione dei comandi utente in messaggi C2S.

```

1 if (strcmp(tok[0], "register") == 0 && ntok == 3) {
2     return send_command(fd, "C2S_REGISTER %s %s", tok[1], tok[2]);
3 }
4 if (strcmp(tok[0], "login") == 0 && ntok == 3) {
5     return send_command(fd, "C2S_LOGIN %s %s", tok[1], tok[2]);
6 }
7 if (strcmp(tok[0], "move") == 0 && ntok == 2 && proto_parse_direction(tok[1], &dir) ==
8     0) {
9     return send_command(fd, "C2S_MOVE %s", proto_direction_name(dir));
10 }
11 if (ntok == 1 && proto_parse_direction(tok[0], &dir) == 0) {
12     return send_command(fd, "C2S_MOVE %s", proto_direction_name(dir));
13 }

```

Lato ricezione, il client interpreta i messaggi S2C e aggiorna una struttura `ui_state_t`. La funzione `ui_render` ridisegna la schermata completa: stato, guida dei comandi, ultimi eventi, mappa locale, mappa globale e prompt.

5.9 Gestione della fine partita

La durata della partita è un timer globale del server. Quando il tempo scade, il server calcola il vincitore in base al numero di celle possedute da ciascun giocatore usato. Il risultato viene inviato a tutti i client ancora connessi con il messaggio:

```
S2C_GAME_OVER <winner> <score> <punteggi>
```

Il punteggio è calcolato scorrendo la matrice `owner`. La funzione `game_build_scores` include tutti gli slot giocatore usati, non solo i giocatori attualmente online; quindi anche un giocatore disconnesso conserva il territorio conquistato prima dell'uscita.

6 Conformità ai requisiti

Requisito	Implementazione	File principali
Client-server TCP	Socket TCP, <code>listen/connect</code> , <code>send/recv</code>	<code>src/common/net.c</code> , <code>src/server/server.c</code> , <code>src/client/client.c</code>
Protocollo applicativo	Messaggi testuali a righe con prefissi <code>C2S</code> e <code>S2C</code>	<code>src/common/protocol.c</code>
Piu' client concorrenti	Multiplexing con <code>select(2)</code> e array dinamico di sessioni	<code>src/server/server.c</code>
Registrazione e login	Database utenti in memoria e autenticazione lato server	<code>src/server/users.c</code> , <code>src/server/server.c</code>
Mappa e conquista celle	Griglia 30x15, matrice muri, matrice proprietà	<code>src/server/game.c</code>
Ostacoli privati	Matrice <code>discovered_walls</code> per ciascun giocatore	<code>src/server/game.h</code> , <code>src/server/game.c</code>
Aggiornamenti periodici	Timer integrato nel timeout di <code>select</code>	<code>src/server/server.c</code>
Timeout e vincitore	Calcolo del vincitore e invio <code>S2C_GAME_OVER</code>	<code>src/server/game.c</code> , <code>src/server/server.c</code>
Interfaccia client	UI testuale con mappe, eventi e comandi rapidi	<code>src/client/ui.c</code> , <code>src/client/client.c</code>
Esecuzione Docker	Immagine Ubuntu e servizi Compose server/client	<code>Dockerfile</code> , <code>compose.yaml</code>

7 Collaudo manuale

Il collaudo previsto è manuale e riproducibile con i comandi forniti nella guida d'uso. Una sequenza minima di verifica è:

1. compilare il progetto con `make`;
2. avviare il server con `./bin/server 4242 60 5`;
3. aprire un primo client con `./bin/client 127.0.0.1 4242`;
4. registrare e autenticare un utente, ad esempio `register alice secret` e `login alice secret`;
5. verificare la comparsa della mappa locale e dell'aggiornamento globale;
6. muoversi con `w/a/s/d` e verificare le risposte `MOVED`, `WALL`, `OUT_OF_BOUNDS` o `OCCUPIED` a seconda dei casi;
7. aprire un secondo client, registrare un secondo utente e verificare lista utenti, ownership delle celle e aggiornamenti globali;
8. attendere la scadenza del timer e verificare il messaggio di fine partita con vincitore e punteggi;
9. ripetere la prova con Docker Compose tramite `docker compose up --build server` e `docker compose run --rm client`.

8 Limiti e possibili estensioni

Il progetto soddisfa i requisiti richiesti, ma alcune scelte sono volutamente semplici:

- gli utenti sono mantenuti solo in memoria;
- le password sono salvate in chiaro;
- la dimensione della mappa e della finestra locale e' compilata nel codice;
- le mappe degli ostacoli sono layout statici inclusi nel sorgente;
- il server usa `select(2)`, quindi il numero massimo di file descriptor gestibili e' vincolato da `FD_SETSIZE` e dai limiti del sistema operativo;
- l'invio verso i client e' sincrono, adeguato a messaggi piccoli e a un contesto didattico.

Estensioni naturali sarebbero la persistenza degli utenti su file, l'hashing delle password, mappe configurabili da file, una gestione esplicita dei pareggi, un protocollo binario con lunghezze prefissate o una versione del server basata su `poll/epoll` per superare alcuni limiti di `select`. Queste estensioni non sono necessarie per il funzionamento attuale, ma individuano direzioni coerenti per un'evoluzione del progetto.

9 Conclusioni

L'applicazione implementa un sistema client-server completo: compilazione separata di server e client, comunicazione TCP, protocollo applicativo definito, gestione multiutente, stato di gioco condiviso, mappe con informazione pubblica e privata, aggiornamenti periodici e fine partita temporizzata. La parte piu' significativa del progetto non e' l'uso di TCP in se', ma la costruzione del livello applicativo sopra uno stream di byte: framing a righe, codifica delle mappe, tokenizzazione, controllo degli errori e sincronizzazione dello stato tra server e client.

Dal punto di vista implementativo, le scelte piu' rilevanti sono il ciclo single-process con `select(2)`, lo stato di gioco centralizzato lato server, gli identificatori stabili dei giocatori e la distinzione tra mappa locale e globale. Queste scelte mantengono il codice relativamente semplice, evitano concorrenza condivisa e permettono di verificare chiaramente il comportamento del protocollo applicativo.